FRAUDRE: Fraud Detection Dual-Resistant to Graph Inconsistency and Imbalance

Ge Zhang*, Jia Wu*, Jian Yang*, Amin Beheshti*, Shan Xue*, Chuan Zhou† and Quan Z. Sheng*
*Department of Computing, Macquarie University, Sydney, Australia
†Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, China
Email: ge.zhang5@students.mq.edu.au,{jia.wu, jian.yang, amin.beheshti, emma.xue}@mq.edu.au,
zhouchuan@amss.ac.cn, michael.sheng@mq.edu.au

2021 IEEE International Conference on Data Mining (ICDM) | 978-1-6654-2398-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICDM51629.2021.00098

***Abstract*—The objective of fraud detection is to distinguish fraudsters from normal users. In graph/network environments, both fraudsters and normal users are modeled as nodes, and the connections between those nodes are represented as edges. Fraudsters typically try to camouflage themselves with "normal" behaviors, say, by deliberately establishing many connections to normal users. Such camouflage inherently makes their appearance inconsistent with the essence of what it is to be normal, and gives rise to inconsistencies in the graph. In this paper, we investigate three aspects of these graph inconsistencies: features, topologies, and relations. To date, graph-based fraud detectors have shown a rather limited capability to fuse information about different types of inconsistencies. Apart from that, there is another problem of imbalance to overcome. This is because fraudsters usually only account for a very small percentage of all users. To achieve a promising capability, *i.e.,* dual-resistant to graph inconsistency and imbalance, we present a new fraud detection model FRAUDRE based on Graph Neural Networks. Extensive experiments comparing eight up-to-date baselines on two real-world datasets, Amazon and YelpChi, demonstrate the superiority of FRAUDRE.**

***Keywords*-fraud detection; anomaly detection; camouflage; fraudster; graph inconsistency; imbalanced learning; graph neural networks; social networks**

I. Introduction

Fraud detection is a critical task in our fight to protect society from financial crime and other behavior with malicious intentions. For instance, opinion fraud in online review platforms can affect the shopping behaviors of their customers [1]–[3], while financial fraud can cause severely damage to our economy [4]–[6]. Rich behavioral interactions can be established among users no matter whether they are normal or fraudulent. These interactions can be transformed into graph-like data through modeling users as nodes and the interactions among them as edges. Benefiting from the superior representation capability of graph structures, graph-based fraud detection has been drawing extensive attention from industry and academia [4], [7]–[13].

To bypass fraud detection systems, fraudsters often camouflage themselves in ways they think to enable them to look "normal" [10], [14]. But their appearance is not consistent with the essence of being normal. Quite the opposite in fact; their presence introduces inconsistencies that can be detected if you know what to look for. As shown in Figure 1, graph inconsistencies can be manifest in three ways:

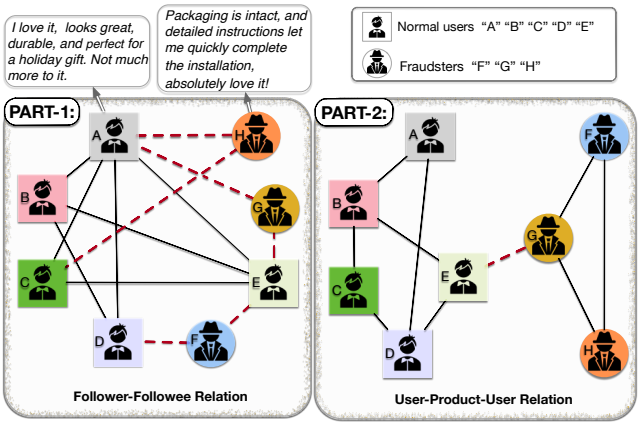


Figure 1. A toy example of the ***graph inconsistency*** issue. Five normal users and three fraudsters were extracted from an online product review platform. As shown in **PART-1** of the figure, users and their comments on products are regarded as nodes and node features, respectively. A graph topology between nodes can be established from the *follower-followee relation*. Under this relation, if one user follows another user in the online review platform, there is an edge between them. ***Feature inconsistency***: Normal users and fraudsters, with different labels, but have similar features. For example, to cover up suspicions, when Fraudster "H" posts a false positive reviews for a product, he imitates the tone of Normal User "A". Therefore, "H" has *inconsistent* feature for its label. ***Topology inconsistency***: Unlike the close connections between normal users, fraudsters have few connections with their accomplices. Here, Fraudsters "F", "G" and "H" have established many connections with normal users but no connections to each other. Beyond the follower-followee relation, as shown in **PART-2**, a new topology between users can be constructed based on the *User-Product-User relation*, where users who have reviewed the same products are connected. ***Relation inconsistency***: Different from the follower-followee relation where fraudsters and normal users are closely connected, under the User-Product-User Relation Fraudsters "F", "G" and "H" only form links with each other and rarely connect with normal users.

feature inconsistency, *topology inconsistency*, and *relation inconsistency*. In addition, fraud detection also faces issues with *graph imbalance*, that is, the highly skewed distribution of fraudsters versus normal users in quantity. This is mainly because, in reality, the number of fraudsters is far less than that of normal users. Taking the real-world dataset Amazon [15] from *Amazon.com* as an example, only 9.5% of users are fraudsters while the others are normal.

Graph neural networks (GNNs) [16]–[18] are currently in focus due to their stellar performance with graph learning tasks, especially node classification. GNNs represent nodes

978-1-6654-2398-4/21/$31.00 ©2021 IEEE
DOI 10.1109/ICDM51629.2021.00098

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

by aggregating neighborhood information. They perform well on graphs that comply with two particular assumptions: first, the homophily assumption, where neighboring nodes have similar features and belong to the same class [19]; and, second, the balance assumption, which requires that the number of samples in different classes is close to equal [20].

Graph inconsistency and imbalance violate the above two assumptions and bring forward four challenges with leveraging GNNs for fraud detection. Feature inconsistency leads to the first challenge, that is, nodes with similar features are likely to belong to different classes. The second challenge is caused by the issue of topology inconsistency where camouflage fraudsters and normal users are located in similar neighborhoods. Because the neighborhood aggregation operation in GNNs will aggregate the features of normal users with those of fraudsters, directly using GNNs for fraud detection will actually make fraudsters more difficult to detect. The third challenge, posed by relation inconsistency, is how to comprehensively integrates node representations/embeddings learned under different relations to improve fraud detection. The fourth challenge is that the loss function in existing GNNs could be dominated by the majority class samples, *i.e.*, normal users. As a result, the model is likely to underfit the minority class samples, *i.e.*, the fraudsters.

To address these challenges and improve graph-based fraud detection, we propose FRAUDRE, which has dual resistance to graph inconsistency and imbalance. FRAUDRE is composed of four modules: 1) a graph-agnostic embedding module, which represents all users by encoding their attribute information; 2) a fraud-aware graph convolution model, which is designed to embed the graph topologies so as to represent the dissimilarity of neighboring users; 3) an intermediate representation combination module concatenates all intermediate embeddings of users to get their final representations; and 4) an imbalance-oriented classification module, which eliminates the training bias towards the large number of normal users. FRAUDRE unifies the above four modules into a GNN-based framework for fraud detection.

Most GNN-based fraud detectors directly apply the neighborhood aggregation operation over the node attributes. As a result, the suspiciousness of fraudsters is greatly reduced at the beginning of model training. This is the reason why the proposed FRADURE leverages the first graph-agnostic embedding module to learn a discriminative representation between normal users and fraudsters. Subsequently, the second fraud-aware graph convolution module further magnifies the differences between normal users and fraudsters.

The main contributions are summarized as follows:

- **Robustness**: FRAUDRE is resistant to the graph inconsistency issue without modifying the original graph topology.
- **Generalisability**: The imbalanced-oriented classification module can be extended to other GNN-based fraud detectors, and the minority class distribution can be adequately modeled with a theoretical guarantee.
- **Effectiveness**: Experiments on two real-world datasets, *i.e*, Amazon and YelpChi demonstrate the effectiveness of FRAUDRE in detecting fraudsters.

II. Related Work

A. Graph Neural Networks

As a powerful graph learning architecture, GNNs have witnessed a growing interest in recent years [16]. Most GNNs adopt a graph convolution operation to aggregate features from the one-hop neighborhoods of target nodes. Through recursive convolution operations, node attributes are propagated multiple hops away on the graph. One of the most representative spectral GNNs, GCN [17], extends the operation of graph convolution in the spectral domain. GraphSAGE [18] proposes an inductive and spatial GNN that aggregates information from a fixed number of sampling nodes. Many complementary variants of GNNs have also been proposed for enhancing graph learning performance, including GAT [21], SGCN [22], and H2GCN [19]. To date, GNNs have been employed in a wide range of downstream application domains, such as recommendation systems [23], community detection [24], and fraud detection [7].

B. Graph-based Fraud Detection

Graph-based fraud detection aims at distinguishing fraudsters from normal users in graph-structured data. Due to the flexibility of GNNs in mining graph data, they have been widely used in detecting fraudsters. For example, SemiGNN [4] proposes a hierarchical attention mechanism to detect the financial fraudsters on Alipay, while MAHINDER [25] is another financial fraudster detector dedicated to forecasting credit risk. However, most of these fraud detection methods apply GNNs directly and so ignore the camouflage behaviors of fraudsters. Because camouflaged fraudsters are such a significant threat, techniques like FdGars [2], GAS [1], and ASA [26] have been proposed. These approaches detect opinion fraud in various mobile applications by pinpointing the camouflage fraudsters. However, their performance is restricted by inflexible detection. For example, ASA [26] extracts attributes from users who have used obvious camouflage behaviors. Subsequently, those inflexible attributes are taken as seeds to detect other fraudsters.

In more recent studies, such as GraphConsis [9], researchers have for the first time been working to investigate the graph inconsistency issue camouflage behaviors causes. To enhance the aggregation process in GNNs, GraphConsis filters dissimilar neighbors for nodes based on a pre-defined threshold. CARE-GNN [10] inherits the neighborhood filtering operation from GraphConsis but adapts a module based on reinforcement learning to make the filtering threshold adaptive. In short, GraphConsis and CARE-GNN detect

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

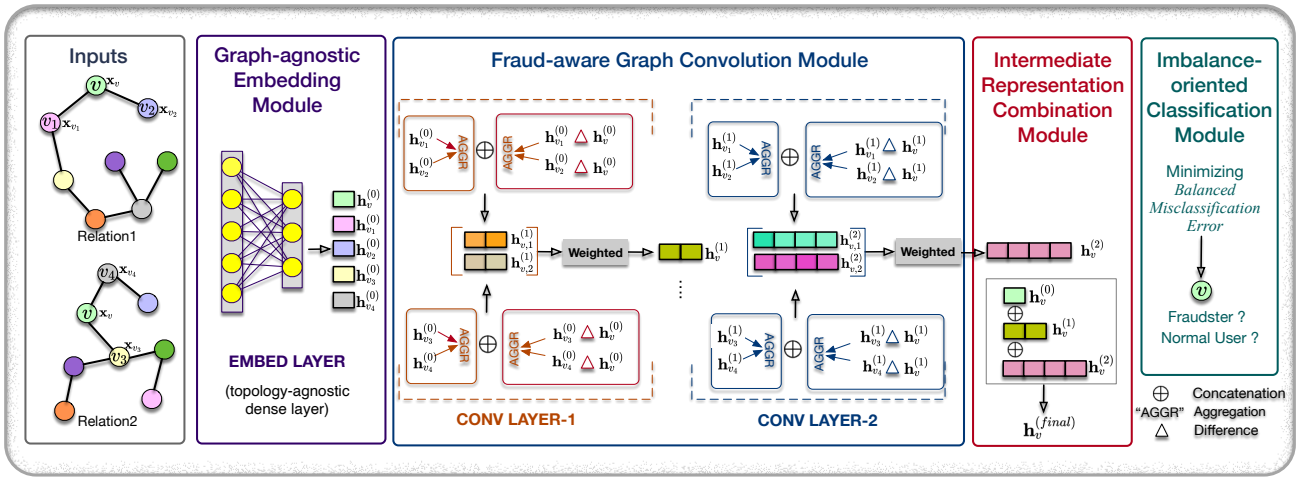


Figure 2. The framework of FRAUDRE. FRAUDRE takes a multi-relation graph with two relations, *i.e.*, Relation1 and Relation2, as the input to predict the label of node v. v_1 and v_2, v_3 and v_4 are v's one-hop neighbors in the topology established by Relation1 and Relation2, respectively. Firstly, the topology-agnostic dense layer (*i.e.*, EMBED LAYER) learns the first-layer embeddings for node v and its one-hop neighbors under two relations. Secondly, under each relation, the fraud-aware graph convolution module learns node embedding $\mathbf{h}_{v,r}^{(l)}$ through two graph convolution layers (*i.e.*, CONV LAYER-1 and CONV LAYER-2), where $r \in \{1,2\}$ and $l \in \{1,2\}$. Specifically, at the end of each graph convolution layer, $\mathbf{h}_v^{(l)}$ is generated via a weighted summation of $\mathbf{h}_{v,1}^{(l)}$ and $\mathbf{h}_{v,2}^{(l)}$. Thirdly, the intermediate representation combination module obtains the final representation $\mathbf{h}_v^{(final)}$ by concatenating $\mathbf{h}_v^{(0)}$, $\mathbf{h}_v^{(1)}$ and $\mathbf{h}_v^{(2)}$ outputted by EMBED LAYER, CONV LAYER-1 and CONV LAYER-2, respectively. By minimizing the *balanced misclassification error*, the class of node v is eventually predicted by the imbalance-oriented classification module.

camouflage fraudsters by modifying the graph structure to meet the homophily assumption GNNs require. However, inputting different graph structures to adjacent graph convolution layer may destabilize GNN training [27].

C. Imbalanced Learning

Due to the significant gap between the number of fraudsters and normal users, fraud detection involves the problem of imbalanced class distributions [28]. In the Euclidean data space, methods investigating imbalanced classification can be roughly categorized into two groups, *i.e.*, data-driven and algorithm-driven [28]. The key idea behind data-driven methods is over-sampling the minority class samples [29], [30] and under-sampling the majority class samples [31]. In contrast, algorithm-driven methods either weight different classes with regard to class frequency [32] or assign different penalties to the classes [33].

Although imbalanced class distribution has been a long-standing issue in deep learning, very few algorithms explore imbalanced class distributions with graphs (*i.e.*, non-Euclidean data) [12]. In classification tasks, most conventional GNNs will overfit the majority class samples and underfit the minorities. DR-GCN [20] is a pioneer work exploring imbalanced class distributions with graphs. Through distribution alignment, DR-GCN forces the class distributions in the training set and the test set to be close. However, DR-GCN cannot scale to large graphs. Another work, GraphSMOTE [34], follows the over-sampling strategy and proposes node and edge generators to address graph imbalance. But such adding operations may degrade robustness of GNN-based methods [35].

III. Preliminaries

A. Definition

Definition 1: (Graph). $\mathcal{G} = \{\mathcal{V}, \mathcal{X}, \mathcal{E}, \mathcal{A}, \mathcal{Y}\}$ represents a graph, where $\mathcal{V} = \{v_i\}_{i=1}^n$ denotes the node set of the graph $\mathcal{G}$. $\boldsymbol{x}_i \in \mathcal{X}$ represents an attribute vector of node v_i and $\boldsymbol{x}_i \in \mathbb{R}^s$. An edge between nodes v_i and v_j is denoted by $e_{ij} \in \mathcal{E}$. The adjacency matrix $\mathcal{A}$ represents the topology of graph $\mathcal{G}$, $\mathcal{A}_{i,j} = 1$, if $e_{ij} \in \mathcal{E}$, 0 otherwise. $y_i \in \mathcal{Y}$ denotes the label of nodes v_i.

Definition 2: (Multi-relation Graph). Different relations can exist among the nodes. A multi-relation graph is represented as $\mathcal{G} = \{\mathcal{V}, \mathcal{X}, \mathcal{E}_1, ..., \mathcal{E}_R, \mathcal{A}_1, ..., \mathcal{A}_R, \mathcal{Y}\}$, where $e_{i,j}^r \in \mathcal{E}_r$ indicates that there is an edge between nodes v_i and v_j under the r-th relation. The topology structures of the multi-relation graph $\mathcal{G}$ can be denoted by $\{\mathcal{A}^r, r \in 1, ..., R\}$, where A^r is an adjacency matrix modeled by $\mathcal{E}_r$, $\mathcal{A}_{i,j}^r = 1$, if $e_{i,j}^r \in \mathcal{E}_r$; otherwise, $\mathcal{A}_{i,j}^r = 0$.

B. Problem Statement

Given a multi-relation graph $\mathcal{G}$, graph-based fraud detector aims to detect the fraudsters with a maximum accuracy performance. Node v_i is associated with the label $y_i \in \{0, 1\}$, where 0 denotes that the node is a normal user and 1 represents a fraudster. Detecting fraudsters in a graph can therefore be regarded as a binary classification task. Only the labels of a subset of the nodes are known. However,

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

through the graph-based fraud detection procedure, unseen fraudsters can be detected.

IV. Overview Framework of FRAUDRE

In the FRAUDRE framework, four different modules are unified into a GNN, with three layers consisting of one topology-agnostic embedding layer and two graph convolution layers. The pipeline of FRAUDRE is shown in Figure 2. The first module, which consists of a topology-agnostic dense layer, learns first-layer representations of the nodes. To further embed rich structure information under different relations, the second module, consisting of two graph convolution layers, aggregates the information from the neighborhood nodes and the neighborhood differences. The third intermediate representation combination module next concatenates all the embeddings output from the first three neural network layers. Finally, the fourth imbalance-oriented classification module detects the fraudsters noting that the class distribution is imbalanced.

V. Fraud detection in the Multi-relation Graph

A. Graph-agnostic Embedding Module

The attributes of fraudsters are similar to normal users in the graph with feature inconsistency. Therefore, it is necessary to learn discriminative embeddings for camouflage fraudsters and normal users at the beginning of model training. We propose a topology-agnostic dense layer to learn the first layer embeddings of nodes. The topology-agnostic dense layer does not rely on any topology information, it only encodes node attributes. The formulation is:

$$\mathbf{h}_i^{(0)} = \sigma\left(\boldsymbol{x}_i \mathbf{W}_1\right), \tag{1}$$

where $\mathbf{x}_i \in \mathbb{R}^s$ is the attribute vector of node v_i, $\mathbf{h}_i^{(0)} \in \mathbb{R}^d$ denotes the intermediate embedding of node v_i, σ is the non-linear activation function ReLU, and $\mathbf{W}_1 \in \mathbb{R}^{s \times d}$ is a learned weight matrix.

After being represented by this module, camouflage fraudsters have dissimilar representations from adjacent normal users. However, by contrast, normal users still maintain a high degree of similarity with most of their neighbors. It is worth emphasizing that most of the neighbors of normal users are still normal users, even if fraudsters have deliberately tried to camouflage themselves by creating connections with many normal users. Fraudsters will typically be a one out of many users. Thus, this module provides a good basis for further magnifying the differences between normal users and fraudsters via topological information.

B. Fraud-aware Graph Convolution Module

The topology inconsistency will mean that the neighborhood aggregation operation in the GNNs will aggregate a large number of features of normal users onto fraudsters, and, if this happens, the fraudsters will be indistinguishable. However, a neighborhood difference aggregation operation based on the discriminative embeddings output by the graph-agnostic embedding module can be introduced to further emphasize the differences among normal users and camouflaged fraudsters.

Since normal users and non-camouflaged fraudsters still meet the homophily assumption, the neighborhood aggregation operation in conventional GNNs needs to be preserved. The graph convolution layer in this module can be formulated as:

$$\begin{aligned}\mathbf{h}_{i,r}^{(l)} = \mathrm{COMBINE}\Big(\mathrm{AGGR}\left\{\mathbf{h}_j^{(l-1)}, v_j \in N_r(i)\right\}, \\ \mathrm{AGGR}\left\{\mathbf{h}_i^{(l-1)} - \mathbf{h}_j^{(l-1)}, v_j \in N_r(i)\right\}\Big),\end{aligned} \tag{2}$$

where $\mathbf{h}_{i,r}^{(l)}$ is the *l*-th layer embedding of node v_i learned under the *r*-th relation, and $l \in 1, ..., L$ and $r \in 1, ..., R$. $N_r(i)$ is the set of 1-hop neighborhoods of node v_i under relation r. "AGGR" and "COMBINE" represent the mean aggregation and the concatenation, respectively.

Because of the relation consistency issue, the contribution of different relations may vary significantly. The *l*-th layer node embedding $\mathbf{h}_i^{(l)}$ is learned through weighted summation,

$$\mathbf{h}_i^{(l)} = \sum_{r=1}^{R} \boldsymbol{\alpha}_r^{(l)} \odot \mathbf{h}_{i,r}^{(l)}, \tag{3}$$

where $\odot$ represents the Hadamard product, and $\boldsymbol{\alpha}_r^{(l)}$ is a learned vector that represents the contribution of relation r to the different embedding dimensions.

C. Intermediate Representation Combination Module

According to the research on graph convolution operations [22], node representations output by different neural network layers have different smoothness. The minority class samples have high energy in the unsmoothed component of node representations. However, the majority class samples, *e.g.*, the normal users, have high energy in the smooth component of node representations [19]. Thus, combining the intermediate representations output by the different neural network layers can further improve fraud detection. In this way, information with different smoothness can be embedded in the final node representations. The formulation is:

$$\mathbf{h}_i^{(final)} = \mathrm{COMBINE}\left(\mathbf{h}_i^{(0)}, \mathbf{h}_i^{(1)}, ..., \mathbf{h}_i^{(L)}\right), \tag{4}$$

where $\mathbf{h}_i^{(final)} \in \mathbb{R}^{(2^{L+1}-1)d}$.

D. Imbalance-oriented Classification Module

Imbalanced class distributions cause most GNN-based fraud detectors to tend to overfit normal users while underfitting fraudsters. This is because most GNN-based fraud detectors adopt the standard cross-entropy loss as the loss function. Here, considering an input is $x \in \mathcal{X}$, a label of $y \in \mathcal{Y}$, and a value of y of 0 or 1. The representation of the standard cross-entropy f^{SCE} would be:

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: FRAUDRE

Input: $\mathcal{G}$: A multi-relation graph; $\mathcal{V}_{train}$: The set of training nodes; $\mathcal{Y}_{train}$: The set of labels of training nodes; L, B, and E: The number of graph convolution layers, training batches, and training epochs.

Output: Vector representations of nodes in $\mathcal{V}_{train}$.

// Initializing model parameters.
for $e \leftarrow 1, ..., E$ **do**
  for $b \leftarrow 1, ..., B$ **do**
    for $l \leftarrow 0, ..., L$ **do**
      if $l == 0$ **then**
        Obtain $\mathbf{h}_i^{(0)}$ by $Eq.(1)$.
      else
        for $r \leftarrow 1, ..., R$ **do**
          Obtain $\mathbf{h}_{i,r}^{(l)}$ by $Eq.(2)$.
        Obtain $\mathbf{h}_i^{(l)}$ by $Eq.(3)$.
    Obtain $\mathbf{h}_i^{(final)}$ by $Eq.(4)$.
    $\mathcal{L} \leftarrow Eq.(12)$
    //Back-propagation
Return $\mathbf{h}_i^{(final)}$, $\forall v_i \in \mathcal{V}_{train}$.

$$
\begin{aligned}
f^{SCE}(y, C(x;\Theta)) &= -\log P(y \mid x) \\
&= -\log \frac{e^{C_y(x;\Theta)}}{\sum_{y'} e^{C_{y'}(x,\Theta)}} \\
&= \log\left[\left(\sum_{y' \neq y} e^{C_{y'}(x;\Theta) - C_y(x;\Theta)}\right) + 1\right],
\end{aligned} \tag{5}
$$

where Θ is the model parameter of the classifier C. f^{SCE} trains C by minimizing the misclassification error. That is,

$$
\sum_{x \in \mathcal{X}} \delta\left(y \neq \text{argmax}_{y'} C_{y'}(x)\right), \tag{6}
$$

where δ is the calculating form of misclassification error.

However, under such a setting, a low misclassification error is still achieved when a fraud detector classifies all samples as normal users [15], [36]. To solve this problem, an alternative is to minimize the average of misclassification error rates in each class [37], formulated as:

$$
\frac{1}{2}\sum_{y=0}^{1}\sum_{x \in \mathcal{X}} \delta'\left(y \neq \text{argmax}_{y'} C_{y'}(x)\right), \tag{7}
$$

where δ' is the calculating form of error rates. Eq. (7) is named as the *balanced misclassification error*. Based on *Theorem* 1 in [15], minimizing the balanced misclassification error is equal to estimating the label y for the input x by supposing:

$$
\text{argmax}_y\, C_y(x;\Theta) = \text{argmax}_y \frac{P(y \mid x)}{P(y)}, \tag{8}
$$

where $P(y)$ are class priors, *i.e.*, the empirical class frequencies of the training set.

This situation is different from the standard cross-entropy loss f^{SCE}. As shown in Eq. (5), f^{SCE} estimates labels of nodes by supposing:

$$
\text{argmax}_y\, C_y(x;\Theta) = \text{argmax}_y\, P(y \mid x). \tag{9}
$$

By reformulating Eq. (8), we get,

$$
\text{argmax}_y\, P(y \mid x) = \text{argmax}_y \left(C_y(x;\Theta) + \log P(y)\right). \tag{10}
$$

Modeling $P(y|x)$ with the softmax, we have the *imbalanced distribution-oriented loss* function f^*, that is,

$$
\begin{aligned}
f^*(y, C(x;\Theta)) &= -\log P(y \mid x) \\
&= -\log \frac{e^{C_y(x;\Theta)+\log P(y)}}{\sum_{y'} e^{C_{y'}(x,\Theta)+\log P(y')}} \\
&= \log\left[\left(\sum_{y' \neq y} \frac{P(y')}{P(y)} e^{C_{y'}(x;\Theta) - C_y(x;\Theta)}\right) + 1\right].
\end{aligned} \tag{11}
$$

A learned weight matrix $\mathbf{W}_2 \in \mathbb{R}^{(2^{L+1}-1)d \times 2}$ is used to complete the final feature transformation on $\mathbf{h}_i^{(final)}$. Based on the newly proposed imbalanced distribution-oriented loss f^*, the loss function of FRAUDRE is defined as:

$$
\mathcal{L} = \sum_{v_i \in \mathcal{V}} f^*(y_i, \mathbf{h}_i^{(final)} \mathbf{W}_2). \tag{12}
$$

The training procedure is presented in Algorithm 1.

VI. Experiments

We conducted a set of experiments on two real-world datasets to investigate the following questions:

- *RQ1*: Does FRAUDRE outperform the up-to-date baseline algorithms?
- *RQ2*: Can FRAUDRE learn discriminative embeddings for normal users and fraudsters?
- *RQ3*: Does FRAUDRE benefit from all four modules?
- *RQ4*: Is FRAUDRE sensitive to the hyper-parameters?
- *RQ5*: How efficient is FRAUDRE to run?

A. Experimental Setup

1) Dataset: We chose the Amazon [38] and YelpChi [39] datasets to validate FRAUDRE's performance. The statistical information of these two datasets is shown in Table I. The first dataset, Amazon, is composed of users and their comments on musical instruments. Users are modeled as nodes and there are three different types of relations among these nodes. Specifically, under the *U-P-U* relation, users

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

Table I
STATISTICAL INFORMATION OF DATASETS

	Nodes (Fraudsters%)	Relation	#Edges	β_r^{normal}	β_r^{fraud}	α_r^{normal}	α_r^{fraud}
Amazon	11,944 (9.5%)	*U-P-U*	175,608	0.8911	0.1673	0.6794	0.6907
		U-S-U	3,566,479	0.9783	0.0558	0.5944	0.5947
		U-V-U	1,036,737	0.9938	0.0532	0.5131	0.5071
		ALL	4,398,392	0.9762	0.0722	0.5404	0.5375
YelpChi	45,954 (14.5%)	*R-U-R*	49,315	0.9982	0.9089	0.9881	0.9511
		R-T-R	573,616	0.8591	0.1764	0.8636	0.8717
		R-S-R	3,402,743	0.8676	0.1857	0.8556	0.8634
		ALL	3,846,979	0.8682	0.1838	0.8549	0.8632

who have commented on one or more same products are linked. The *U-S-U* relation links users who issued at least one rating (1-5 stars) that was the same within 7 days. Under the last relation, *U-V-U*, users who have comments ranked in the top 5% in terms of TF-IDF similarity are linked. In addition, all topologies under different relations are merged together to form the relation *ALL* in Table I. Users with more than 80% or less than 20% helpful votes are labeled as normal users and fraudsters, respectively. Similarly, the YelpChi dataset includes legitimate and spam reviews on restaurants and hotels. Reviews are taken as nodes and there are also three type of relations among nodes. To be specific, under the *R-U-R* relation, reviews provided by the same user are linked. For the *R-S-R* relation, if two reviews give the same rating on an item, they will be linked. Under the *R-T-R* relation, links are constructed between the reviews on the same item issued in the same month.

2) Baselines:

- GCN [17]: This spectral GNN represents nodes via extending the graph convolution in the spectral domain.
- GAT [21]: This method employs an attention mechanism and builds a graph attention neural network to enhance embedding performance.
- GraphSAGE [18]: This is a spatial GNN method that represents nodes inductively based on the nodes' ego features and the aggregated neighborhood features.
- GEM [5]: This detector is built from a GCN and is dedicated to malicious account detection.
- FdGars [2]: Similar to GEM, FdGars is also GCN-based but aimed at spam detection.
- Player2Vec [11]: This GCN-based fraud detector is dedicated to heterogeneous networks.
- GraphConsis [9]: This GNN-based fraud detector considers graph inconsistency and is achieved by filtering dissimilar neighbors for nodes based on a fixed threshold.
- CARE-GNN [10]: This method is camouflage-resistant. It filters neighbors for nodes by an adaptive threshold to enhance the graph convolution process.

Among these baselines, GCN, GAT, GraphSAGE, and FdGars target graphs characterized by *Definition 1* (*i.e.*, *ALL* in Table I), while GEM, Play2Vec, GraphConsis, CARE-GNN, and FRAUDRE run on multi-relation graphs.

3) Experimental Settings and Implementation: In FRAUDRE, we set the embedding size in the graph-agnostic embedding module to 64, the number of graph convolution layers to 2, batch size to 256 (Amazon) and 1024 (YelpChi). For model optimization, we adopt the widely-used Adam [40] optimizer and set the learning rate to 0.1 for Amazon and 0.001 for YelpChi.

GCN, GAT, and GraphSAGE were implemented using the source codes released by their corresponding authors. And, we implemented GEM, FdGars, Player2Vec, GraphConsis, and CARE-GNN through an open-source toolbox for fraud detection[1]. The code for FRAUDRE and the datasets are available at GitHub[2]. ALL experiments were conducted by Python 3.7, 1 NVIDIA Volta GPUs, and 384G DRAM.

4) Evaluation Metrics: We adopted AUC, Macro-Recall, and Macro-F1 as evaluation metrics. AUC denotes the prediction probability that selects a fraudster randomly, and its ranking is higher than that of a normal user. Macro-Recall measures the unweighted mean of the proportions of true fraudsters and true normal users that a detector detected from the total number of ground truth fraudsters and normal users respectively. F1-score is the trade-off between recall and precision, and Macro-F1 is an unweighted mean of the F1-score for normal users and fraudsters.

B. The Evidence of Graph Inconsistency and Imbalance

Firstly, we validated the existence of topology and relation inconsistency. Two metrics were employed – β_r^{fraud} and β_r^{normal}, which represented the average label similarity of the fraudsters and the normal users with their one-hop neighbors under the relation r. The formulations were:

$$\beta_r^{fraud} = \frac{1}{|\{v_i|y_i=1\}|} \sum_{v_i \in fraudsters} \frac{|\{v_j|v_j \in N_r(i) \& y_i == y_j\}|}{|\{v_j|v_j \in N_r(i)\}|}, \tag{13}$$

and

$$\beta_r^{normal} = \frac{1}{|\{v_i|y_i=0\}|} \sum_{v_i \in normal\ users} \frac{|\{v_j|v_j \in N_r(i) \& y_i == y_j\}|}{|\{v_j|v_j \in N_r(i)\}|}. \tag{14}$$

A low average label similarity score β_r^{fraud} ($\rightarrow$ 0) for the fraudsters implies that the fraudsters have successfully camouflaged themselves in the topology under the relation r, *i.e.*, that most of the one-hop neighbors to the fraudsters are normal users. On the other hand, a high score β_r^{fraud} ($\rightarrow$ 1) implies that fraudsters have largely failed to camouflage themselves and are densely connected with their accomplices.

- *Topology Inconsistency*. As shown in Table I, almost all β_r^{fraud} scores for both datasets were lower than

[1]https://github.com/safe-graph/DGFraud
[2]https://github.com/FraudDetection/FRAUDRE

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

Table II
PERFORMANCE (%) ON AMAZON UNDER DIFFERENT PERCENTAGE OF TRAINING DATA. RECALL (ABBR. MACRO-RECALL) F1 (ABBR. MACRO-F1)

Data	Method	10%			20%			30%			40%		
		AUC	Recall	F1	AUC	Recall	F1	AUC	Recall	F1	AUC	Recall	F1
Amazon	**GCN [17]**	77.26	50.00	47.51	77.42	50.00	47.51	76.99	50.00	47.51	77.94	50.00	47.51
	GAT [21]	76.96	50.00	47.50	76.99	50.00	47.51	76.61	50.00	47.50	77.35	50.00	47.50
	GraphSAGE [18]	69.87	50.00	47.50	71.75	50.00	47.50	72.51	50.00	47.51	71.49	50.00	47.50
	GEM [5]	70.34	69.56	75.52	72.05	71.55	70.53	73.99	72.12	68.48	74.44	70.66	74.42
	FdGars [2]	81.10	73.41	55.32	81.19	73.47	55.31	80.91	72.90	55.01	80.82	72.82	55.14
	Player2Vec [11]	83.89	50.00	47.51	83.89	50.00	47.51	86.53	50.00	47.50	85.45	50.00	47.50
	GraphConsis [9]	82.67	82.63	75.97	84.22	84.21	81.93	84.46	84.37	79.06	85.15	85.10	77.98
	CARE-GNN [10]	88.16	88.19	88.21	88.25	87.95	85.80	87.41	84.89	75.70	87.36	83.90	88.36
	FRAUDRE(ours)	**90.37**	**89.12**	**91.02**	**88.99**	**88.71**	**90.67**	**91.51**	**88.01**	**91.11**	**88.18**	**88.61**	**91.10**

Table III
PERFORMANCE (%) ON YELPCHI UNDER DIFFERENT PERCENTAGE OF TRAINING DATA. RECALL (ABBR. MACRO-RECALL) F1 (ABBR. MACRO-F1)

Data	Method	10%			20%			30%			40%		
		AUC	Recall	F1	AUC	Recall	F1	AUC	Recall	F1	AUC	Recall	F1
YelpChi	**GCN [17]**	52.12	50.00	46.08	53.88	50.00	46.08	52.62	50.00	46.08	53.12	50.00	46.08
	GAT [21]	50.14	50.00	46.08	49.94	50.00	46.08	49.97	50.00	46.08	49.67	50.00	46.08
	GraphSAGE [18]	52.94	50.00	46.08	55.39	50.00	46.08	56.08	50.00	46.10	56.45	50.00	46.10
	GEM [5]	64.35	50.00	46.08	64.28	51.33	48.89	69.63	51.04	48.24	70.88	50.38	46.87
	FdGars [2]	47.36	49.19	48.76	47.54	49.40	48.93	47.71	49.52	49.02	47.91	49.42	48.93
	Player2Vec [11]	50.43	50.00	46.08	51.11	50.00	46.08	50.88	50.00	46.08	51.44	50.00	46.08
	GraphConsis [9]	64.12	64.72	**61.30**	63.89	64.46	**62.93**	60.94	61.44	**62.73**	61.02	61.67	**63.03**
	CARE-GNN [10]	69.73	65.68	52.86	70.47	66.94	57.55	72.42	67.32	57.39	70.99	66.80	56.47
	FRAUDRE(ours)	**72.21**	**66.44**	55.34	**72.51**	**67.30**	58.22	**73.72**	**67.91**	59.24	**72.22**	**66.98**	59.26

0.2 (with the exception of *R-U-R* in YelpChi), which implies that the fraudsters have a large number of connections with normal users with regard to these relations. The β_r^{normal} score for normal users was higher than 0.85 under all relations. Thus, the connections among normal users were always very dense regardless of the relations.

- *Relation Inconsistency*. As illustrated in Table I, the fraudsters failed to camouflage themselves in *R-U-R* in YelpChi (the β_r^{fraud} under *R-U-R* was 0.9089), but they were successful with the other two relations, *R-T-R* and *R-S-R* (a β_r^{fraud} under *R-T-R* and *R-S-R* of 0.1764 and 0.1857, respectively).

Secondly, we analyzed the *feature inconsistency*. In this step, we adopted two metrics, α_r^{fraud} and α_r^{normal}, for measuring the average feature similarity among neighboring nodes. We especially investigate the neighboring nodes due to the fact that most of the one-hop neighbors of fraudsters are normal users except the topology based on *R-U-R* in YelpChi. Two metrics are formulated as:

$$\alpha_r^{fraud} = \frac{1}{|\{v_i|y_i=1\}|} \sum_{v_i \in fraudsters} \frac{\sum_{v_j \in N_r(i)} Similarity(x_i, x_j)}{|\{v_j|v_j \in N_r(i)\}|}, \quad (15)$$

and

$$\alpha_r^{normal} = \frac{1}{|\{v_i|y_i=0\}|} \sum_{v_i \in normal\ users} \frac{\sum_{v_j \in N_r(i)} Similarity(x_i, x_j)}{|\{v_j|v_j \in N_r(i)\}|}, \quad (16)$$

where cosine similarity was used as the similarity measurement.

- *Feature Inconsistency*. As shown in Table I, the attributes of the fraudsters were similar to the normal users neighboring them. For example, α_r^{fraud} was 0.8717 and 0.8634 under *R-T-R* and *R-S-R*, respectively. In these two relations, most one-hop neighbors to the fraudsters were normal users. To be noticed, normal users retained high similarity with the normal users neighboring with them.

In terms of the graph imbalance issue, only 9.5% and 14.5% of nodes were fraudsters in Amazon and YelpChi, respectively.

C. Performance Comparison (RQ1)

The experimental results of the proposed FRADURE and the baselines on Amazon and YelpChi are reported in Tables II and III, respectively. We varied the percentage of the training samples from 10% to 40% and observed that FRAUDRE outperformed all eight baselines under most training proportions with regard to the metrics. Further analysis of the experiment results follows:

1) Multi-relation vs. Single-relation. Recall that GCN, GAT, GraphSAGE, and FdGars run on graphs with single relations (*ALL* in Table I), while GEM, Player2Vec, GraphConsis, CARE-GNN, and FRAUDRE aim to detect fraudsters in the multi-relation graphs. As shown in the Amazon results in Table II, Player2Vec and GEM did not perform better than the algorithms designed for the single

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

Table IV
THE RESULT OF REPRESENTATION SIMILARITY ANALYSIS. WE EXPERIMENT ON FRAUDRE WITH "EMBED LAYER" AND "CONV LAYER-1".

(a) Statistical Results on Amazon.

Nodes	Relation	Average Cosine Similarity			
		Attribute	*M1*	*M2*	*P2inM2*
Fraudsters	*U-P-U*	0.6907	0.0690	0.0689	0.0566
	U-S-U	0.5947	0.0621	0.0618	0.0219
	U-V-U	0.5071	0.0639	0.0639	0.0346
Normal users	*U-P-U*	0.6794	0.6181	0.6169	0.5104
	U-S-U	0.5944	0.5330	0.5500	0.5383
	U-V-U	0.5131	0.5671	0.5603	0.5851

(b) Statistical Results on YelpChi.

Nodes	Relation	Average Cosine Similarity			
		Attribute	*M1*	*M2*	*P2inM2*
Fraudsters	*R-U-R*	0.9511	0.2023	0.2198	0.1921
	R-T-R	0.8717	0.1019	0.1019	0.0934
	R-S-R	0.8634	0.1019	0.1018	0.0965
Normal users	*R-U-R*	0.9811	0.8653	0.8653	0.8672
	R-T-R	0.8636	0.8852	0.8652	0.8552
	R-S-R	0.8556	0.8290	0.8290	0.8475

relation graph, *e.g.*, FdGars. The reason is that directly applying GNNs to a fraud detection task in a multi-relation environment is not feasible. To this end, GraphConsis and CARE-GNN both achieved promising results because they filter dissimilar neighbors based on node attributes before the GNN aggregation operation. FRAUDRE, with its four modules, outperformed all other baselines on the both single- and multi-relation graphs.

2) Detecting Camouflage Fraudsters. GraphConsis, CARE-GNN, and FRAUDRE all detected the camouflage behavior of the fraudsters and all outperformed the algorithms that could not make this distinction. FdGars, which also considered camouflage behavior, performed well on Amazon, while still has much room for improvement on YelpChi. This is because FdGars only considers camouflaged fraudsters who share features similar to labeled fraudsters when, in fact, there are many camouflaged fraudsters with features divergent to the fraudsters that are labeled. GraphConsis and CARE-GNN adopted more flexible ways of detecting camouflage fraudsters, so they outperformed FdGars. FRAUDRE yielded better results in terms of AUC and Recall than GraphConsis and outperformed CARE-GNN in terms of all evaluation metrics.

3) Recall Capability. Since the paper is focused on fraud detection, and both datasets, Amazon and Yelpchi, are imbalanced, recall under the macro setting is an important metric for evaluating the performance of all methods. As shown in Tables II and III, the recall scores for GCN, GAT, GraphSAGE, and Play2Vec were only around 50%. That is to say, these algorithms classified all fraudsters as normal users. With these algorithms, no measures are taken to alleviate classification bias towards the majority class samples. Moreover, directly aggregating the information of normal users with fraudsters exacerbates the bias. GraphConsis and CARE-GNN had successfully recall of fraudsters on both datasets. The reason is that these two algorithms under-sample the normal users to equalize the number of normal users and fraudsters in the training set. In addition, they filter dissimilar neighbors for each node before the neighborhood aggregation operation. Our model achieves the best performance on two datasets. *w.r.t.* recall.

D. Representation Similarity Analysis (RQ2)

To demonstrate that FRAUDRE can learn discriminative embeddings for normal users and fraudsters, we evaluated the average representation similarity among neighboring nodes. Tables IV(a) and IV(b) show the average cosine similarity between fraudsters and their one-hop neighbors, and normal users and their one-hop neighbors under each relation on Amazon and YelpChi, respectively. *Attribute* denotes the attributes of nodes. *M1* denotes the intermediate embedding output by the graph-agnostic embedding module, *M2* denotes the intermediate embedding output by the fraud-aware convolution module, and *P2inM2* denote the intermediate embedding output by the neighborhood difference aggregation operation in the fraud-aware convolution module.

In case where fraudsters establish many connections with normal users, for example, *U-S-U* on Amazon, the average cosine similarity of original attributes between the fraudsters and their neighbors was 0.5947. However, after training FRAUDRE, the average cosine similarity of *M1*, *M2*, and *P2inM2* between the fraudsters and their one-hop neighbors was only 0.0621, 0.0618, 0.0219, respectively. However, for normal users, the average cosine similarity of original attributes between them and their one-hop neighbors under *U-S-U* was 0.5944. Further, the average cosine similarity of *M1*, *M2*, and *P2inM2* between the normal users and their neighbors remained at the high level (around 0.55). Other relations, *U-P-U* and *U-V-U* on Amazon and *R-T-R* and *R-S-R* on YelpChi, under which fraudsters also successfully camouflage themselves, show a similar trend. Even if fraudsters under the relation *R-U-R* in YelpChi fail to camouflage themselves topologically, the average similarity of *M1*, *M2*, and *P2inM2* between the fraudsters and their neighbors was still at a low level. A possible reason is that the behavior of fraudsters is not as stable as that of normal users, and the difference among fraudsters may significant.

The above analysis implies that the graph-agnostic embedding module is able to recognize the feature inconsistency in graphs - that it can learn the discriminative embeddings for normal users and fraudsters at the beginning of model

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

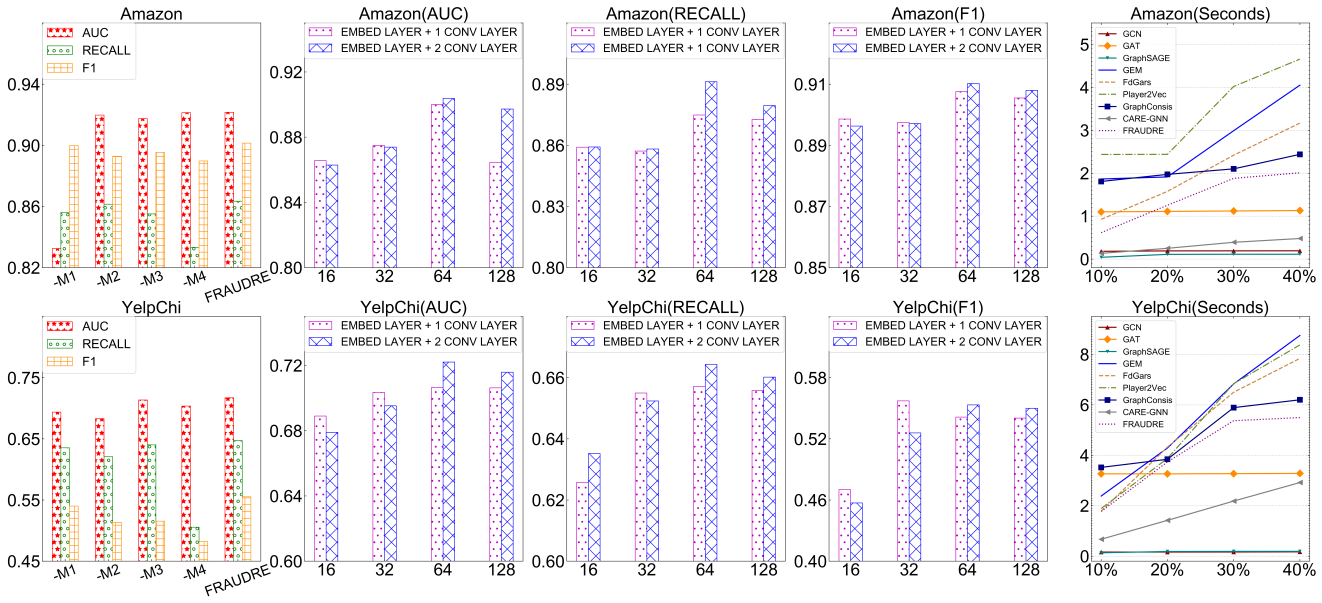


Figure 3. Ablation Analysis (Column 1), Parameter Analysis (Columns 2, 3, and 4), and Time Efficiency Analysis (Column 5) on Amazon and YelpChi.

learning. It also shows that the difference in aggregation operations in the fraud-aware convolution module can magnify the differences between normal users and fraudsters.

E. Ablation Analysis (RQ3)

To demonstrate the effectiveness of individual components in FRAUDRE, we conducted a series of ablation studies. The four variants tested were: (i) FRAUDRE-M1: which did not include the graph-agnostic embedding module so the graph convolution operation was performed on the node attributes directly; (ii) FRAUDRE-M2: where only the neighbor aggregation operation was preserved in the fraud-aware graph convolution module; (iii) FRAUDRE-M3: in which only the embedding output by last graph convolution layer was taken as the final node embeddings; and (iv) FRAUDRE-M4: which used the standard cross-entropy loss to classify nodes instead of the imbalanced distribution-oriented loss.

As shown in Figure 3 (Column 1), FRAUDRE was superior to the above four variants *w.r.t.* all evaluation metrics. In particular, the performance of FRAUDRE-M4 with the standard cross-entropy loss greatly under-performed the other variants *w.r.t.* Recall. This indicates that the imbalanced distribution-oriented loss function in the imbalance-oriented classification module effectively improves the generalization ability of the classifier on the minority class samples (*i.e.*, fraudsters).

F. Parameter Sensitivity (RQ4)

We also investigated the sensitivity of FRAUDRE to the parameters with regard to the number of graph convolution layers and different embedding sizes in the graph-agnostic embedding module. The AUC, Macro-Recall, and Macro-F1 results for both Amazon and YelpChi are shown on the top and bottom of Figure 3 (Columns 2, 3, and 4), respectively. The results demonstrate that our method maintains acceptable results with most hyper-parameters combinations and has relative stability.

G. Time Efficiency (RQ5)

To demonstrate the run-time performance of all comparison algorithms, we recorded the average training time of all algorithms per epoch with the proportion of training data varying from 10% to 40%. For all algorithms, we set the number of graph convolution layers to 1, the embedding size to 64, and the batch size to 256 on Amazon, versus 1024 for YelpChi. As shown in Figure 3 (Column 5), FRAUDRE has obvious advantages in running speed compared to the fraud detection algorithms designed for single relation graphs (*e.g.*, FdGars). FRAUDRE also runs faster than GraphConsis which is designed for multi-relation graphs.

VII. Conclusion

In this paper, we aimed to tackle the graph inconsistency and imbalance issues in the face of camouflaged fraudsters. Our proposed fraud detector, FRAUDRE, is built from a GNN and consists of four modules: a graph-agnostic embedding module, a fraud-aware graph convolution module, an intermediate representation combination module, and an imbalance-oriented classification module. These modules, unified into a GNN, learn discriminative embeddings for both normal users and fraudsters which reduces the training bias towards normal users. Extensive experiments on two real-world datasets, *i.e.,* Amazon and YelpChi, demonstrate the superior performance of FRAUDRE in terms of AUC,

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.

Macro-recall, and Macro-F1 scores against eight up-to-date baseline methods.

ACKNOWLEDGMENT

This work was supported by the ARC DECRA Project (No. DE200100964) and the ARC Discovery Project (No. DP200102298).

REFERENCES

[1] A. Li, Z. Qin, R. Liu, Y. Yang, and D. Li, "Spam review detection with graph convolutional networks," in *CIKM*, 2019, pp. 2703–2711.

[2] J. Wang, R. Wen, C. Wu, Y. Huang, and J. Xion, "Fdgars: Fraudster detection via graph convolutional networks in online app review system," in *WWW*, 2019, pp. 310–316.

[3] G. Zhang, L. Zhao, J. Huang, J. Wu, C. Zhou, and J. Yang, "efraudcom: An e-commerce fraud detection system via competitive graph neural networks," *ACM Trans. Inf. Syst.*, 2021.

[4] D. Wang, J. Lin, P. Cui, Q. Jia, Z. Wang, Y. Fang, Q. Yu, J. Zhou, S. Yang, and Y. Qi, "A semi-supervised graph attentive network for financial fraud detection," in *ICDM*, 2019, pp. 598–607.

[5] Z. Liu, C. Chen, X. Yang, J. Zhou, X. Li, and L. Song, "Heterogeneous graph neural networks for malicious account detection," in *CIKM*, 2018, pp. 2077–2085.

[6] C. Liu, Q. Zhong, X. Ao, L. Sun, W. Lin, J. Feng, Q. He, and J. Tang, "Fraud transactions detection via behavior tree with local intention calibration," in *KDD*, 2020, pp. 3035–3043.

[7] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," *Decis. Support Syst.*, vol. 133, p. 113303, 2020.

[8] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: a survey," *Data Min. Knowl. Discov.*, vol. 29, no. 3, pp. 626–688, 2015.

[9] Z. Liu, Y. Dou, P. S. Yu, Y. Deng, and H. Peng, "Alleviating the inconsistency problem of applying graph neural network to fraud detection," in *SIGIR*, 2020, pp. 1569–1572.

[10] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *CIKM*, 2020, pp. 315–324.

[11] Y. Zhang, Y. Fan, Y. Ye, L. Zhao, and C. Shi, "Key player identification in underground forums over attributed heterogeneous information network embedding framework," in *CIKM*, 2019, pp. 549–558.

[12] X. Ma, J. Wu, S. Xue, J. Yang, C. Zhou, Q. Z. Sheng, H. Xiong, and L. Akoglu, "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, 2021.

[13] H. Wang, C. Zhou, J. Wu, W. Dang, X. Zhu, and J. Wang, "Deep structure learning for fraud detection," in *ICDM*, 2018, pp. 567–576.

[14] B. Hooi, H. A. Song, A. Beutel, N. Shah, K. Shin, and C. Faloutsos, "Fraudar: Bounding graph fraud in the face of camouflage," in *KDD*, 2016, pp. 895–904.

[15] G. Collell, D. Prelec, and K. Patil, "Reviving threshold-moving: a simple plug-in bagging ensemble for binary and multiclass imbalanced data," *arXiv*, 2016.

[16] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.

[17] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017.

[18] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *NeurIPS*, 2017, pp. 1025–1035.

[19] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," *NeurIPS*, 2020.

[20] M. Shi, Y. Tang, X. Zhu, D. Wilson, and J. Liu, "Multi-class imbalanced graph convolutional network learning," in *IJCAI*, 2020.

[21] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," *ICLR*, 2018.

[22] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *ICML*, 2019, pp. 6861–6871.

[23] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Trans. Knowl. Data Eng.*, 2020.

[24] X. Su, S. Xue, F. Liu, J. Wu, J. Yang, C. Zhou, W. Hu, C. Paris, S. Nepal, D. Jin *et al.*, "A comprehensive survey on community detection with deep learning," *arXiv*, 2021.

[25] Q. Zhong, Y. Liu, X. Ao, B. Hu, J. Feng, J. Tang, and Q. He, "Financial defaulter detection on online credit payment via multi-view attributed heterogeneous information network," in *WWW*, 2020, pp. 785–795.

[26] R. Wen, J. Wang, C. Wu, and J. Xiong, "Asa: Adversary situation awareness via heterogeneous graph convolutional networks," in *WWW*, 2020, pp. 674–678.

[27] X. Zhang and M. Zitnik, "Gnnguard: Defending graph neural networks against adversarial attacks," in *NeurIPS*, 2020.

[28] K. Cao, C. Wei, A. Gaidon, N. Arechiga, and T. Ma, "Learning imbalanced datasets with label-distribution-aware margin loss," in *NeurIPS*, 2019.

[29] J. Byrd and Z. Lipton, "What is the effect of importance weighting in deep learning?" in *ICML*, 2019, pp. 872–881.

[30] S. Ando and C. Y. Huang, "Deep over-sampling framework for classifying imbalanced data," in *ECML-PKDD*, 2017, pp. 770–785.

[31] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, 2009.

[32] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *CVPR*, 2019, pp. 9268–9277.

[33] Z.-H. Zhou and X.-Y. Liu, "Training cost-sensitive neural networks with methods addressing the class imbalance problem," *IEEE Trans. Knowl. Data Eng.*, vol. 18, no. 1, pp. 63–77, 2005.

[34] T. Zhao, X. Zhang, and S. Wang, "Graphsmote: Imbalanced node classification on graphs with graph neural networks," in *WSDM*, 2021, pp. 833–841.

[35] D. Zügner, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on neural networks for graph data," in *KDD*, 2018, pp. 2847–2856.

[36] H. Lin, Y. Lu, X. Han, and L. Sun, "Adaptive scaling for sparse detection in information extraction," *arXiv*, 2018.

[37] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, "Long-tail learning via logit adjustment," in *ICLR*, 2020.

[38] J. J. McAuley and J. Leskovec, "From amateurs to connoisseurs: modeling the evolution of user expertise through online reviews," in *WWW*, 2013, pp. 897–908.

[39] S. Rayana and L. Akoglu, "Collective opinion spam detection: Bridging review networks and metadata," in *KDD*, 2015, pp. 985–994.

[40] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *ICLR*, 2015.

Authorized licensed use limited to: Macquarie University. Downloaded on November 05,2026 at 08:22:54 UTC from IEEE Xplore. Restrictions apply.